

Описание языка программирования Обольд

1. Введение

Словарь Обольда состоит из идентификаторов, чисел, строк, операторов, разделителей и комментариев, которые называются лексемами и состоят из последовательностей литер. Пробелы и переносы не должны встречаться внутри лексем (исключая комментарии и пробелы в строках). Они игнорируются, если они не существенны для отделения двух последовательных лексем. Заглавные и строчные буквы считаются различными.

2. Словарь

буква = "A" | "B" | ... | "Z" | "a" | ... | "z"
цифра = "0" | "1" | ... | "9"
шестнадцатеричнаяЦифра = цифра | "A" | ... | "F" | "a" | ... | "f"

Идентификаторы — это последовательности букв и цифр. Первая литера идентификатора должна быть буквой.

идентификатор = буква { буква | цифра | "_" }

Числа — это знаковые целые или вещественные числа. Целые числа являются последовательностью цифр и могут следовать за префиксом. Если префикса нет, то представление десятичное. Префикс "0x" указывает на шестнадцатеричное представление.

Вещественное число всегда содержит десятичную точку. Опционально оно может также содержать десятичный порядок. Буква E означает «умножить на десять в степени».

число = ["+" | "-"] целое | вещественное
целое = цифра { цифра } | "0x" шестнадцатеричнаяЦифра { шестнадцатеричнаяЦифра }
вещественное = цифра { цифра } ("." { цифра } [порядок] | порядок)
порядок = ("e" | "E") ["+" | "-"] цифра { цифра }

Строки — это последовательность литер, заключенных в двойные кавычки. Сами кавычки могут использоваться в строке после обратной косой черты. Число литер в строке называется длиной строки.

строка = " " { элементСтроки } " "
элементСтроки = неЭкранированныйСимвол | экранированныйСимвол
неЭкранированныйСимвол = " " | "!" | "#" | ... | "[" | "]" | ... | "~"
экранированныйСимвол = "\" | "\\ | \"\n\" | \"\t\"

Зарезервированные последовательности литер не могут использоваться в качестве идентификаторов: switch case if else while do elif for fn module return var type const true false is in nil div mod struct

Комментарии могут быть вставлены между любыми двумя лексемами в программе. Они являются произвольными последовательностями литер, которые открываются скобкой `/*` и закрываются с помощью `*/`, либо следуют за `//` до конца строки, при условии что `//` не находится внутри строкового литерала. Комментарии не влияют на смысл программы.

3. Объявления и области видимости

Каждый идентификатор, встречающийся в программе, должен быть объявлен заранее, если это не предопределенный идентификатор. Объявления также служат для задания определенных постоянных свойств объекта, например, является ли он константой, типом, переменной или процедурой.

Позднее идентификатор используется для ссылки на соответствующий объект. Это возможно только в тех частях программы, которые находятся в пределах области объявления. Идентификатор не может обозначать больше чем один объект внутри данной области. Область распространяется текстуально от места объявления до конца блока, к которому объявление принадлежит.

Идентификатор, объявленный в блоке модуля, может сопровождаться меткой экспорта `"export"`, чтобы указать, что он экспортируется из определяющего модуля. В этом случае, идентификатор может быть использован в других модулях, если они импортируют объявляющий модуль. В таком случае, идентификатор предваряется идентификатором, обозначающим его модуль. Префикс и идентификатор разделены точкой и вместе называются уточненным идентификатором.

уточненныйИдентификатор = [идентификатор "."] идентификатор
объявлениеИдентификатора = [export] идентификатор

4. Объявления констант

Объявление константы связывает ее идентификатор с ее значением.

объявлениеКонстанты = объявлениеИдентификатора "=" константноеВыражение
константноеВыражение = выражение

Константное выражение должно быть вычислимым при транслировании текста программы без ее выполнения. Операнды константного выражения также должны являться константами.

5. Объявления типов

Тип данных определяет множество значений, которые могут принимать переменные этого типа, и применимые операции. Объявление типа используется для связи идентификатора с типом.

объявлениеТипа = объявлениеИдентификатора ":" тип
тип = уточненныйИдентификатор | типМассив | типСтруктура
| типУказатель | процедурныйТип

Элементарные типы

Следующие элементарные типы обозначаются предопределенными идентификаторами.

bool	логические значения true и false
char	литеры
byte	8-битное целое число без знака
i64	64-битное целое число со знаком
f64	64-битное число с плавающей точкой
set	набор целых чисел между 0 и 63

Типы массив

Массив — это структура, состоящая из фиксированного числа элементов одинакового типа, называемого типом элементов. Количество элементов массива называется его длиной. Элементы массива обозначаются индексами, которые являются целыми числами от 0 до длины массива минус 1. Размерности указываются в порядке от внешней к внутренней.

```
типМассив = тип "[ " длина " ] " { "[ " длина " ] " }
длина     = константноеВыражение
```

```
matrix: i64[10][20]
vector: f64[10]
```

Типы структура

Тип структура состоит из фиксированного числа элементов, которые могут иметь различные типы. Объявление типа структура задает для каждого элемента, называемого полем, его тип и идентификатор, который обозначает это поле.

Если тип структура экспортируется, идентификаторы полей, которые должны быть видимыми вне модуля объявления, должны быть помечены. Они называются публичными полями; не отмеченные поля называются приватными полями. Структуры являются расширяемыми, т.е. структура может быть определена как расширение другой структуры.

```
типСтруктура      = struct [ " ( " базовыйТип " ) " ] " { " [ множествоСписковПолей ] " } "
базовыйТип        = уточненныйИдентификатор
множествоСписковПолей = списокПолей { списокПолей }
списокПолей       = списокИдентификаторов " : " тип " ; "
списокИдентификаторов = объявлениеИдентификатора { " , " объявлениеИдентификатора }
```

```
Point: struct {
  x, y: i64;
}

Pixel: struct (Point) {
  color: i64;
}
```

Типы указатель

Переменные типа указатель R принимают в качестве значений указатели на переменные некоторого типа T. Этот тип должен быть структурой. Тип указатель R называется связанным с типом T, а T — является базовым типом указателя R. Типы указатель наследуют отношение расширения

базовых типов, если они есть. Если тип T является расширением T_0 и P является указателем, связанным с T , то тогда P также является расширением P_0 , который является указателем, связанным с T_0 .

типУказатель = "*" тип

```
Tree: *Node
```

Процедурные типы

Переменные процедурного типа T принимают в качестве значения процедуру или NIL . Если процедура P связана с переменной процедурного типа T , то типы формальных параметров процедуры P должны совпадать с типами соответствующих формальных параметров типа T . То же самое справедливо для типа результата.

процедурныйТип = сигнатураПроцедуры
сигнатураПроцедуры = формальныеПараметры "→" возвращаемыйТип

```
BinaryOp: (x, y: i64) -> i64  
Display: (n: i64) -> void
```

6. Объявления переменных

Объявления переменных служат для введения переменных и связывания их с идентификаторами и типами данных, которые должны быть уникальными в пределах заданной области определения.

объявлениеПеременных = списокИдентификаторов ":" тип

```
i, j, k: i64  
p, q: bool  
a: f64[100]
```

7. Выражения

Выражения — это конструкции, которые задают правила вычисления значений. Выражения состоят из операндов и операций.

Операнды

За исключением чисел и строк, операнды определяются обозначениями. Обозначение может быть идентификатором константы, переменной или процедуры. Этот идентификатор может быть уточнен идентификатором модуля и сопровождаться селекторами, если обозначенный объект — часть структуры.

Если p обозначает переменную-указатель, $*p$ обозначает переменную, на которую ссылается p . Если g обозначает запись, то $g.f$ обозначает поле f из записи g . Если p обозначает указатель, $p.f$ обозначает поле f записи $*p$, то есть точка подразумевает разыменование, и $p.f$ означает $(*p).f$.

Охрана типа $v(T_0)$ обеспечивает, чтобы v имел тип T_0 , то есть охрана типа прекращает выполнение программы, если v имеет тип отличный от T_0 . Охрана применима, если T_0 является расширением объявленного типа T в v , и если v - ссылка на структуру, или v - указатель.

Если обозначенный объект является переменной, то обозначение ссылается на текущее значение переменной. Если объект является процедурой, обозначение без списка параметров ссылается на эту процедуру. Если за обозначением следует список параметров, обозначение подразумевает возвращаемый результат её исполнения.

обозначение = ["*"] уточненныйИдентификатор { селектор }

селектор = "." идентификатор | "[" выражение "]" | "(" уточненныйИдентификатор ")"

```
i
*a[i]
w[3].ch
t.left.right
t(CenterNode).subnode
```

Операции

В синтаксисе выражений различаются четыре класса операций с различными приоритетами. Операция **!** имеет наивысший приоритет, за которой следуют мультипликативные операции, аддитивные операции и отношения. Операции одного приоритета выполняются слева направо.

выражение = простоеВыражение [отношение простоеВыражение]

отношение = "==" | "!=" | "<" | "<=" | ">" | ">=" | "is" | "in"

простоеВыражение = ["+" | "-"] слагаемое { операцияСложения слагаемое }

операцияСложения = "+" | "-" | "|" | "

слагаемое = множитель { операцияУмножения множитель }

операцияУмножения = "*" | "/" | "div" | "mod" | "&&"

множитель = число | строка | nil | true | false | множество | обозначение | вызовПроцедуры | "(" выражение ")" | "!" множитель

множество = "{ " [элемент " , " элемент] " }

элемент = выражение ["." . " выражение]

Логические операции

Эти операции применяются к операндам `bool` и дают результат `bool`.

Лексема Результат

| | логическая дизъюнкция

& & логическая конъюнкция

! логическое отрицание

Арифметические операции

Операции **+**, **-**, ***** и **/** применяются к операндам числовых типов. Оба операнда должны быть одного типа, что также определяет тип результата. При использовании в качестве унарных операций **-** обозначает инверсию знака, а **+** обозначает операцию идентичности.

Операции `div` и `mod` применяются только к целочисленным операндам. Пусть $q = x \text{ div } y$ и $r = x \text{ mod } y$. Тогда q и r определяются уравнением

$$X = q * y + r, 0 \leq r < y$$

Лексема	Результат
+	сумма
−	разность
*	произведение
/	вещественное деление
<code>div</code>	деление нацело
<code>mod</code>	остаток

Операции над множествами

Лексема	Результат
+	объединение
−	разность
*	пересечение
/	симметрическая разность

Когда используется с одним операндом типа `set`, знак минус обозначает дополнение множества.

Отношения

Отношения являются логическими операциями. Отношение упорядочения `<`, `<=`, `>`, `>=` применяется к числовым типам, `char` и литерным массивам. Отношения `==` и `!=` применимы также к типам `bool`, `set`, указателям и процедурным типам.

`v is T` означает «`v` имеет тип `T`». Это применимо, если `T` — расширение объявленного типа `T0` для `v`, и `v` — указатель.

Лексема	Отношение
<code>==</code>	равно
<code>!=</code>	неравно
<code><</code>	меньше
<code><=</code>	меньше или равно
<code>></code>	больше
<code>>=</code>	больше или равно
<code>is</code>	проверка типа
<code>in</code>	наличие в множестве

8. Операторы

Операторы обозначают действия. Есть элементарные и структурные операторы. Элементарные операторы не состоят из каких-либо частей, которые сами являлись бы операторами. Это такие операторы как присваивание и вызов процедуры. Структурные операторы состоят из частей, которые сами являются операторами.

оператор	=	[операторПрисваивания операторВызоваПроцедуры операторIf операторSwitch операторWhile операторDoWhile операторFor]
последовательностьОператоров	=	{ оператор }

Присваивание

Присваивание служит для замены текущего значения переменной на новое значение, заданное выражением.

Если параметр-значение процедуры структурирован, присваивание ему или его элементам не допускается. Для импортированных переменных также не допускаются присваивания.

Тип выражения должен быть таким же, как у обозначений. Имеют место следующие исключения:

1. Константу `nil` можно присвоить переменным любого типа указателя или процедуры.
2. Строки могут быть присвоены любому массиву литер, если количество литер в строке меньше, чем количество литер в массиве (добавляется нулевая литера). Строки, состоящие из одной литеры, также могут быть присвоены переменным типа `char`.
3. В случае записей тип источника должен быть расширением типа адресата.
4. Открытый массив может быть присвоен массиву равного базового типа.

операторПрисваивания	=	присваивание ";"
присваивание	=	обозначение "=" выражение

```
i = 0
k = (i + j) div 2
f = log2
w[i+1].ch = "A"
```

Вызов процедуры

Вызов процедуры активирует процедуру. Он может содержать список фактических параметров, которые заменяют соответствующие формальные параметры. Имеются два вида параметров: параметры-ссылки и параметры-значения. В случае параметров-ссылок, фактический параметр должен быть обозначением, обозначающим переменную. Если параметр является параметром-значением, соответствующий фактический параметр должен быть выражением.

операторВызоваПроцедуры	=	вызовПроцедуры ";"
вызовПроцедуры	=	обозначение фактическиеПараметры
фактическиеПараметры	=	" (" [списокВыражений] ") "
списокВыражений	=	выражение { ", " выражение }

```
ReadInt(i)
WriteInt(2*j + 1, 6)
INC(w[k].count)
```

Оператор If

Операторы If определяют условное выполнение операторов. Логическое выражение, предшествующее последовательности операторов, называется условием. Если условие окажется равным `true`, выполняется связанная с этим условием последовательность операторов. Иначе выполняется последовательность операторов, записанная после слова `else`, если оно имеется.

операторIf = "if" выражение телоУсловия ["else" (операторIf | телоУсловия)]
телоУсловия = "{ " последовательностьОператоров " } "

```
if ch >= "A" && ch <= "Z" {  
    ReadIdentifier();  
} else if ch >= "0" && ch <= "9" {  
    ReadNumber();  
} else {  
    ReadString();  
}
```

Оператор Switch

Оператор Switch определяет выбор и выполнение последовательности операторов по значению выражения. Сначала вычисляется выбирающее выражение, а затем выполняется та последовательность операторов, чей список меток варианта содержит полученное значение.

Тип `T` оценочного выражения также может быть структурой или указателем. Тогда метки должны быть расширениями `T`.

операторSwitch = switch выражение "{ " вариант { вариант } " } "
вариант = "case" списокМетокВарианта " : " последовательностьОператоров
списокМетокВарианта = меткиВарианта { " , " меткиВарианта }
меткиВарианта = метка [" . . " метка]
метка = целое | строка | уточненныйИдентификатор

```
switch k {  
    case 0: x = x + y;  
    case 1: x = x - y;  
    case 2: x = x * y;  
    case 3: x = x / y;  
}  
  
switch p {  
    case P0: p.b = 10;  
    case P1: p.b = 2.5;  
    case P2: p.b = true;  
}
```

Операторы циклов

Оператор `while` задает повторное выполнение. Условия вычисляются и выполнение продолжается, пока хотя бы одно из выражений равняется `true`.

Оператор `do ... while` задает повторное выполнение последовательности операторов пока условие равняется `true`. Выполняется по крайней мере один раз.

Оператор `for` определяет повторное выполнение последовательности операторов фиксированное число раз для прогрессии значений целочисленной переменной. Типы `beg` и `end` должны

быть `i64`, и `inc` должно быть целым константным выражением. Если шаг не определен, то он предполагается равным 1.

операторWhile = `while` выражение телоУсловия `{ "else if" выражение телоУсловия " }`
операторDoWhile = `"do" телоУсловия "while" выражение " ; "`
операторFor = `"for" " ( " идентификатор "=" выражение " , " выражение`
`[ " , " константноеВыражение ] " ) " телоУсловия`

```
while j > 0 {
    j = j / 2;
    i = i+1;
}

while t != nil && t.key != i {
    t = t.left;
}

while m > n {
    m = m - n;
} elif n > m {
    n = n - m;
}

do {
    k = k / 2;
    n = n + 1;
} while k != 0;

for (v = beg, end, inc) {S}
```

9. Объявления процедур

Объявление процедуры состоит из заголовка процедуры и тела процедуры. Заголовок определяет имя процедуры, формальные параметры и тип результата. Тело содержит объявления и операторы.

Процедуры могут возвращать или не возвращать значение. В последнем случае возвращаемый тип помечается как `void`. Процедуры активизируются вызовом процедуры. Если процедура возвращает значение, она должна содержать оператор возврата `return`, который определяет ее результат.

Все константы, переменные и типы, объявленные внутри тела процедуры, локальны в процедуре. Объявленные глобально объекты также видны в процедуре.

объявлениеПроцедуры	=	заголовокПроцедуры телоПроцедуры
заголовокПроцедуры	=	"fn" объявлениеИдентификатора сигнатураПроцедуры
телоПроцедуры	=	последовательностьОбъявлений "{ "[последовательностьОператоров] ["return" выражение] " } "
последовательностьОбъявлений	=	[блокОбъявленияКонстант] [блокОбъявленияТипов] [блокОбъявленияПеременных]
блокОбъявленияКонстант	=	"const" телоБлокаКонстант
блокОбъявленияТипов	=	"type" телоБлокаТипов
блокОбъявленияПеременных	=	"var" телоБлокаПеременных
телоБлокаКонстант	=	объявлениеКонстанты "{ " объявлениеКонстанты "; " { объявлениеКонстанты "; " } " }
телоБлокаТипов	=	объявлениеТипа "{ " объявлениеТипа "; " { объявлениеТипа "; " } " }
телоБлокаПеременных	=	объявлениеПеременных "{ " объявлениеПеременных "; " { объявлениеПеременных "; " } " }

Формальные параметры

Формальные параметры — это идентификаторы фактических параметров, которые будут конкретизированы при вызове процедуры. Имеются два вида параметров: параметры-значения и параметры-ссылки. Параметры-ссылки соответствуют переменным и их означают. Параметр-ссылка обозначается лексемой & перед типом. Параметр-значение соответствует значению выражения, которое не может быть изменено присвоением. Однако, если параметр-значение имеет элементарный тип, то он представляет локальную переменную, к которой изначально присвоено значение фактического выражения. Формальные параметры локальны для процедуры.

формальныеПараметры	=	" (" [секцияФормальногоПараметра { " , " секцияФормальногоПараметра }] ") "
возвращаемыйТип	=	уточненныйИдентификатор типУказатель процедурныйТип "void"
секцияФормальногоПараметра	=	идентификатор { " , " идентификатор } " : " ["&"] формальныйТип
формальныйТип	=	уточненныйИдентификатор { " [] " }

Тип каждого формального параметра определен в списке параметров. Для параметров-ссылок он должен быть идентичным соответствующему типу фактического параметра, кроме структур, когда он должен быть базовым типом соответствующего фактического параметра.

Если тип формального параметра определен как T [] то такой параметр называется открытым массивом, и соответствующий фактический параметр может быть произвольной длины.

Если формальный параметр определен как процедурный тип, то соответствующий фактический параметр должен быть либо глобальной процедурой, либо переменной процедурного типа. Фактический параметр не может быть предопределенной процедурой. Тип результата процедуры не может быть ни структурой, ни массивом.

```

fn SafeDiv(x, y: i64, res: &i64) -> bool
var isCorrect: bool;
{
  if y == 0 {
    isCorrect = false;
  } else {
    res = x / y;
    isCorrect = true;
  }
  return isCorrect;
}

```

```

fn WriteInt(x: i64) -> void
var {i: i64; buf: i64[5];}
{
  i = 0;
  do {
    buf[i] = x mod 10;
    x = x div 10;
    i = i + 1;
  } while x == 0;
  do {
    i = i - 1;
    Write(chr(buf[i] + ord("0")));
  } while i == 0;
}

```

10. Модули

Модуль — совокупность объявлений констант, типов, переменных и процедур вместе с последовательностью операторов, предназначенных для присваивания начальных значений переменным. Модуль обычно представляет собой единицу компиляции.

модуль	=	"module" идентификатор [списокИмпорта] последовательностьОбъявлений { объявлениеПроцедуры }
списокИмпорта	=	"import" импорт { ", " импорт } "; "
импорт	=	[идентификатор "="] идентификатор

В списке импорта указаны модули, клиентом которых является модуль. Если идентификатор x экспортируется из модуля M , и если M указан в списке импорта модуля, тогда обращение к x осуществляется как $M.x$. Если в списке импорта используется форма « $M = M1$ », экспортируемый объект x , объявленный в $M1$, вызывается в импортирующем модуле как $M.x$.

Идентификаторы, которые должны быть экспортированы отмечаются словом `export` в их объявлении. Переменные всегда экспортируются в режиме только для чтения.

Процедура с названием `init` выполняется, когда модуль загружается в систему. Она не должна принимать аргументов и не должна возвращать какие-либо значения.

```

module Out
import Texts, Oberon;

var W: Texts.Writer;

fn export Write(ch: char) -> void {
    Texts.Write(W, ch);
}

fn export WriteInt(x, n: i64) -> void
var {i: i64, a: char[16];}
{
    // логика
}

fn export WriteLn() -> void {
    Texts.WriteLn(W);
    Texts.Append(Oberon.Log, W.buf);
}

fn init() -> void {
    Texts.OpenWriter(W);
}

```

11. Синтаксис Обольда

буква	= "A" "B" ... "Z" "a" ... "z"
цифра	= "0" "1" ... "9"
шестнадцатеричнаяЦифра	= цифра "A" ... "F" "a" ... "f"
идентификатор	= буква { буква цифра "_" }
число	= ["+" "-"] целое вещественное
целое	= цифра { цифра } "0x" шестнадцатеричнаяЦифра { шестнадцатеричнаяЦифра }
вещественное	= цифра { цифра } ("." { цифра } [порядок] порядок)
порядок	= ("e" "E") ["+" "-"] цифра { цифра }
строка	= " " { элементСтроки } " "
элементСтроки	= неЭкранированныйСимвол экранированныйСимвол
неЭкранированныйСимвол	= " " "!" "#" ... "[" "]" ... "~"
экранированныйСимвол	= "\" "\\ \"\n \"\t
уточненныйИдентификатор	= [идентификатор "."] идентификатор
объявлениеИдентификатора	= [export] идентификатор
объявлениеКонстанты	= объявлениеИдентификатора "=" константноеВыражение
константноеВыражение	= выражение
объявлениеТипа	= объявлениеИдентификатора ":" тип
тип	= уточненныйИдентификатор типМассив типСтруктура типУказатель процедурныйТип
типМассив	= тип "[" длина "]" { "[" длина "]" }
длина	= константноеВыражение
типСтруктура	= struct "[" (" базовыйТип ") "]" { "[" множествоСписковПолей "]" }
базовыйТип	= уточненныйИдентификатор
множествоСписковПолей	= списокПолей { списокПолей }
списокПолей	= списокИдентификаторов ":" тип ";"

списокИдентификаторов	= объявлениеИдентификатора { " , " объявлениеИдентификатора }
типУказатель	= " * " тип
процедурныйТип	= сигнатураПроцедуры
сигнатураПроцедуры	= формальныеПараметры " -> " возвращаемыйТип
объявлениеПеременных	= списокИдентификаторов " : " тип
обозначение	= [" * "] уточненныйИдентификатор { селектор }
селектор	= " . " идентификатор " [" выражение "] "
	" (" уточненныйИдентификатор ") "
выражение	= простоеВыражение [отношение простоеВыражение]
отношение	= " == " " != " " < " " <= " " > " " >= " " is " " in "
простоеВыражение	= [" + " " - "] слагаемое { операцияСложения слагаемое }
операцияСложения	= " + " " - " " " "
слагаемое	= множитель { операцияУмножения множитель }
операцияУмножения	= " * " " / " " div " " mod " " && "
множитель	= число строка nil true false множество обозначение вызовПроцедуры
множество	= " { " [элемент " , " элемент] " } "
элемент	= выражение [" . . " выражение]
	" (" выражение ") " " ! " множитель
оператор	= [операторПрисваивания операторВызоваПроцедуры
	операторIf операторSwitch операторWhile
	операторDoWhile операторFor]
последовательностьОператоров	= { оператор }
операторПрисваивания	= присваивание " ; "
присваивание	= обозначение " = " выражение
операторВызоваПроцедуры	= вызовПроцедуры " ; "
вызовПроцедуры	= обозначение [фактическиеПараметры]
фактическиеПараметры	= " (" [списокВыражений] ") "
списокВыражений	= выражение { " , " выражение }
операторIf	= " if " выражение телоУсловия [" else " (операторIf телоУсловия)]
телоУсловия	= " { " последовательностьОператоров " } "
операторSwitch	= switch выражение " { " вариант { вариант } " } "
вариант	= " case " списокМетокВарианта " : " последовательностьОператоров
списокМетокВарианта	= меткиВарианта { " , " меткиВарианта }
меткиВарианта	= метка [" . . " метка]
метка	= целое строка уточненныйИдентификатор
операторWhile	= while выражение телоУсловия
	{ " else if " выражение телоУсловия " } "
операторDoWhile	= " do " телоУсловия " while " выражение " ; "
операторFor	= " for " " (" идентификатор " = " выражение " , " выражение
	[" , " константноеВыражение] ") " телоУсловия
объявлениеПроцедуры	= заголовокПроцедуры телоПроцедуры
заголовокПроцедуры	= " fn " объявлениеИдентификатора сигнатураПроцедуры
телоПроцедуры	= последовательностьОбъявлений " { "
	[последовательностьОператоров] [" return " выражение] " } "
последовательностьОбъявлений	= [блокОбъявленияКонстант]
	[блокОбъявленияТипов]
	[блокОбъявленияПеременных]
блокОбъявленияКонстант	= " const " телоБлокаКонстант
блокОбъявленияТипов	= " type " телоБлокаТипов
блокОбъявленияПеременных	= " var " телоБлокаПеременных

телоБлокаКонстант	=	объявлениеКонстанты "{ " объявлениеКонстанты "; "
		{ объявлениеКонстанты "; " } " }
телоБлокаТипов	=	объявлениеТипа "{ " объявлениеТипа "; "
		{ объявлениеТипа "; " } " }
телоБлокаПеременных	=	объявлениеПеременных "{ " объявлениеПеременных "; "
		{ объявлениеПеременных "; " } " }
формальныеПараметры	=	" (" [секцияФормальногоПараметра { ", "
		секцияФормальногоПараметра }] ") "
возвращаемыйТип	=	уточненныйИдентификатор
		типУказатель процедурныйТип "void"
секцияФормальногоПараметра	=	идентификатор { ", " идентификатор } " : " ["&"] формальныйТип
формальныйТип	=	уточненныйИдентификатор { " [] " }
модуль	=	"module" идентификатор [списокИмпорта]
		последовательностьОбъявлений { объявлениеПроцедуры }
списокИмпорта	=	"import" импорт { ", " импорт } "; "
импорт	=	[идентификатор "="] идентификатор